1 Lane2 Language Specification

Version: v1 draft

Status: working specification

Contents

1.1 Introduction

Lane2 is a strict, pure, expression-oriented functional programming language. Its first implementation target is a parser, a type checker, and an AST interpreter; later implementations may add bytecode compilation, a virtual machine, a linker, and effect handling.

Lane2 is intentionally small. It has no mutable state, assignment, trait system, method dispatch, module system, or implicit typeclass-style instance search in v1. Instead, v1 is centered on nominal data, first-class functions, bidirectional local type inference, exhaustive pattern matching, and explicit operation values opened into lexical scope.

This document specifies Lane2/Core v1: the platform-independent part of Lane2 that should behave the same across the initial AST interpreter and later execution backends.

1.1.1 Scope

Lane2/Core v1 covers:

- source structure and declarations;
- lexical scope and name resolution;

- nominal struct and enum types;
- function and block expressions;
- local type inference;
- pattern matching;
- `open`, `preopen`, and operation-based operators;
- unsafe builtin expressions.

The following are outside Lane2/Core v1:

- mutation and assignment;
- modules and imports;
- bytecode and virtual machine semantics;
- linker entrypoint selection;
- algebraic effects;
- type aliases;
- traits, typeclasses, and interfaces;
- tuples and collection syntax.

1.1.2 Compatibility

This specification is a v1 draft. No compatibility is promised between draft revisions. Language rules may be added, removed, or changed while the first implementation is being developed.

Once a v1 implementation is declared stable, this section should be replaced by an explicit compatibility policy.

1.1.3 Experimental Features

This draft does not distinguish stable and experimental language features. Every rule in this document is provisional until v1 is stabilized.

Future drafts may mark individual features as experimental when their surface syntax or semantics are intentionally unsettled.

1.1.4 Feedback

Design feedback is tracked in the Lane2 repository. Implementation changes should update this specification, `docs/language-v1.md`, `docs/prelude-v1.md`, `CONTEXT.md`, and ADRs when the change affects user-visible semantics.

1.1.5 Reference

When referencing this draft, use:

> Lane2 Language Specification : Lane2/Core v1 draft.

1.2 Conformance

The words “must”, “must not”, “may”, and “should” are normative in this document.

Text marked as rationale, examples, or notes is informative unless it explicitly uses normative language.

An implementation conforms to this draft if it accepts all valid programs described here, rejects invalid programs described here, and gives the specified typing and evaluation behavior for safe programs.

Programs that misuse `builtin` are outside Lane2′s safety guarantee.

1.3 Notation

Grammar fragments in this document use an EBNF-like notation.

- Terminal sequences are written in single quotes, as in `'fn'`.
- Lexical categories are written in angle brackets, as in `<identifier>`.
- Non-terminals are written in lowercase camel case, as in `sourceFile`.
- A sequence `A B` means `A` followed by `B`.
- A choice `A | B` means either `A` or `B`.
- An optional item is written `[A]`.
- A repeated item is written `{A}`.

- Parentheses group grammar items.

The grammar fragments are intended to specify accepted source shape. The initial parser may follow the MoonBit parser's precedence and associativity where this specification has not deliberately removed a MoonBit feature.

1.4 Lexical Overview

Lane2 is Unicode source text. The exact Unicode identifier profile is implementation-defined for the first draft, but identifiers must not collide with keywords or operator tokens.

Whitespace and comments separate tokens and are otherwise insignificant except where needed to disambiguate tokens.

Line comments start with `//` and continue to the end of the line. Delimited comments start with `/*` and end with `*/`.

In type annotation syntax, the colon between a value or field name and a type must have whitespace on both sides:

```
let answer : Int = 42
fn add(a : Int, b : Int) -> Int { a + b }
struct Point {
  x : Int
  y : Int
}
```

This spacing rule applies to type annotations only. Struct literal field assignment remains `field: expression`:

```
Point::{ x: 1, y: 2 }
```

V1 keywords:

```
struct enum fn let open if else match builtin
```

V1 reserved words for future use:

```
effect handler module import pub type trait interface mut return
```

Reserved words are not valid ordinary identifiers.

The concrete tokenization of operators, precedence, and associativity should follow the MoonBit parser reference where Lane2 keeps the corresponding syntax.

1.5 Core Semantics

Lane2 v1 is pure and strict.

- Evaluating a safe expression depends only on its inputs and produces a value.
- Function arguments are evaluated before the function body runs.
- `if` and `match` evaluate only the selected branch.
- There is no mutable binding, mutable field, assignment, field update, or implicit statement sequencing.
- IO and other effects are not part of v1 core semantics.

The `Unit` value is written explicitly as `()`.

Empty blocks are invalid. A block must contain exactly one final expression after any local items.

1.6 Source Files

A Lane2 source file is a sequence of top-level definitions.

There is no language-level `main` entrypoint in v1. Entrypoint selection belongs to a later linker or runtime layer.

Top-level definitions are keyword-delimited. Lane2 source does not use MoonBit `///|` separators or semicolon-delimited top-level items.

```
sourceFile:
    { topLevelDefinition }
```

```
topLevelDefinition:
    structDeclaration
  | enumDeclaration
  | functionDeclaration
  | topLevelLet
  | anonymousTopLevelLet
  | openDeclaration
```

Top-level forms:

```
struct Point {
  x : Int
  y : Int
}

enum Option[A] {
  none
  some(A)
}

fn add(a : Int, b : Int) -> Int {
  a + b
}

let answer : Int = 42

let : Add[Int] = Add::{ add: int_add }

open int_add_ops
```

1.7 Top-Level Scope

Top-level type and function definitions form recursive definition groups.

Top-level `struct`, `enum`, and `fn` definitions may refer to each other regardless of textual order.

Top-level value definitions and top-level `open` declarations follow ordered value scope:

- a top-level `let` initializer may refer only to values already available at that declaration point;
- a top-level `open` may open only a value already available at that declaration point;
- a top-level function body may refer to any top-level value or function, even one declared later.

Example:

```
fn read_x() -> Int {
  x
}

let x : Int = 1
```

This is valid because the function body sees the complete top-level environment.

```
let y : Int = x
let x : Int = 1
```

This is invalid because top-level value initializers follow ordered value scope.

1.8 Namespaces

Lane2 has separate type and value namespaces.

Type and value names may use the same spelling:

```
enum Option[A] {
  none
  some(A)
}
```

```
let Option : Int = 42
```

Syntax of the form `Type::member` resolves `Type` in the type namespace.

```
let x : Option[Int] = Option::some(1)
```

The `Option` on the left of `::` denotes the enum type, not the value named `Option`.

1.9 Primitive Types

V1 primitive types:

```
Int
Bool
String
Unit
```

There is no `Float`, `Char`, tuple type, array type, list type, map type, or collection literal in v1.

`()` is the only `Unit` value.

1.10 Nominal Data Types

User-defined data is nominal.

```
struct UserId {
  value : Int
}

struct OrderId {
  value : Int
}
```

`UserId` and `OrderId` are distinct types even though their fields match structurally.

Generic type definitions put type parameters after the type name:

```
struct Box[A] {
  value : A
}

enum Option[A] {
  none
  some(A)
}
```

Generic type application uses brackets:

```
Box[Int]
Option[String]
```

There are no type aliases in v1.

```
type:
    typeName [ typeArguments ]
  | functionType

typeArguments:
    '[' type { ',' type } [ ',' ] ']'

functionType : [ typeParameters ] '(' [ type { ',' type } [ ',' ] ] ')' '->' type

typeParameters:
    '[' typeParameter { ',' typeParameter } [ ',' ] ']'
```

1.11 Structs

```
structDeclaration:
    'struct' typeName [ typeParameters ] '{' { structMember } '}'
```

```
structMember:
    fieldDeclaration
  | fieldForwarding

fieldDeclaration:
    fieldName ':' type

fieldForwarding:
    'open' fieldName

structLiteral:
    typeName [ typeArguments ] '::' '{' structLiteralField { ',' structLiteralField } [ ',' ]
'}'

structLiteralField:
    fieldName
  | fieldName ':' expression
```

Struct literals are qualified:

```
let p : Point = Point::{ x: 1, y: 2 }
```

Struct field punning is allowed:

```
let p : Point = Point::{ x, y }
```

This is equivalent to:

```
let p : Point = Point::{ x: x, y: y }
```

Struct literals must provide every field exactly once.

The following are not supported:

- anonymous record literals,
- default fields,
- spread,
- copy update,
- field update.

Fields are accessed with dot syntax:

```
p.x
```

V1 has no field visibility modifiers. Every struct field is accessible wherever the struct value is accessible.

1.12 Enums

```
enumDeclaration:
    'enum' typeName [ typeParameters ] '{' { enumVariant } '}'

enumVariant:
    variantName [ '(' [ type { ',' type } [ ',' ] ] ')' ]

qualifiedVariantExpression:
    typeName [ typeArguments ] '::' variantName [ '(' [ expression { ',' expression } [ ',' ] ]
')' ]

unqualifiedVariantExpression:
    variantName [ '(' [ expression { ',' expression } [ ',' ] ] ')' ]
```

Enum variants are scoped to their enum.

Variant names may be lowercase or uppercase. Capitalization has no semantic role.

Payloadless variants are values and are written without call parentheses:

```
let x : Option[Int] = Option::none
```

This is invalid:

```
Option::none()
```

Variant payloads are positional:

```
enum Expr {
  int(Int)
  add(Expr, Expr)
}
```

Labeled variant payloads are not part of v1. Named product data must be represented with a struct:

```
struct Node[A] {
  left : Tree[A]
  value : A
  right : Tree[A]
}

enum Tree[A] {
  leaf(A)
  node(Node[A])
}
```

In expressions, unqualified variants are allowed when unambiguous:

```
let x : Option[Int] = some(1)
```

If more than one visible enum has a variant named `some`, the unqualified expression is invalid unless another directly local rule disambiguates it. A qualified variant is always allowed:

```
let x : Option[Int] = Option::some(1)
```

In patterns, variants must always be qualified.

1.13 Functions

Functions are uncurried. There is no automatic currying or partial application.

Function definitions use block bodies:

```
functionDeclaration:
    'fn' [ typeParameters ] functionName '(' [ parameter { ',' parameter } [ ',' ] ] ')' '->'
type block

parameter:
    valueName ':' type

functionLiteral:
    'fn' [ typeParameters ] '(' [ functionLiteralParameter { ',' functionLiteralParameter }
[ ',' ] ] ')' [ '->' type ] block

functionLiteralParameter:
    valueName
  | valueName ':' type
```

```
fn add(a : Int, b : Int) -> Int {
  a + b
}
```

There is no `= expr` function body form.

Function result types use `->`, not `:`.

All named functions must state every parameter type and the result type.

Generic named functions place type parameters after `fn` and before the name:

```
fn[A] id(value : A) -> A {
  value
}
```

Function parameters are positional in v1. Labeled function parameters are not supported.

Function types use a parenthesized parameter list:

```
() -> Int
(Int) -> Int
(Int, Int) -> Int
```

`Int -> Int` is not a valid one-argument function type.

Generic function types place type parameters before the value parameter list:

```
[A](A) -> A
[A](Bool, A, A) -> A
```

Function literals produce function values:

```
let f : (Int, Int) -> Int = fn(a, b) {
  a + b
}
```

Generic function literals place type parameters after `fn`:

```
let id = fn[A](value : A) {
  value
}
```

1.14 Blocks

A block expression contains local items followed by one final expression.

```
block:
    '{' { localItem } expression '}'

localItem:
    localLet
  | localFunctionDeclaration
  | openDeclaration

localLet:
    'let' valueName [ ':' type ] '=' expression

topLevelLet:
    'let' valueName ':' type '=' expression

anonymousTopLevelLet:
    'let' ':' type '=' expression

openDeclaration:
    'open' valueName
```

```
{
  let x = 1
  fn double(n : Int) -> Int {
    n + n
  }
  double(x)
}
```

Allowed local items:

- `let`,
- named `fn`,
- `open`.

Local type definitions are not supported in v1.

A block does not contain expression statements. This is invalid:

```
{
  f(x)
  g(y)
}
```

An empty block is invalid. Write `()` explicitly when a `Unit` value is required.

1.15 Local Bindings

Local `let` bindings are sequential. A local binding is visible only to later items and the final expression in the same block.

Local `let` bindings may omit type annotations when their initializer can synthesize a type by local type inference:

```
let x = 1
```

Local named functions are also sequential. They may call themselves, but local mutually recursive groups and forward references are not supported:

```
fn f(n : Int) -> Int {
  fn loop(x : Int) -> Int {
    if x == 0 {
      0
    } else {
      loop(x - 1)
    }
  }
  loop(n)
}
```

Local value names may shadow earlier local value names.

1.16 Local Type Inference

Lane2 uses bidirectional local type inference.

Type information may:

- synthesize upward from an expression,
- check downward from an expected type.

Type information moves between adjacent syntax nodes. The type checker does not create global Hindley-Milner constraints and does not infer a binding's type from later uses.

Function literals may be checked against an expected function type:

```
let f : (Int, Int) -> Int = fn(a, b) {
  a + b
}
```

Without an expected type, a function literal must provide explicit value parameter types:

```
let f = fn(a : Int, b : Int) {
  a + b
}
```

Generic function literals without an expected type must provide explicit type parameters and explicit value parameter types:

```
let id = fn[A](value : A) {
  value
}
```

This is invalid:

```
let id = fn[A](value) {
  value
}
```

Polymorphic calls and generic data constructors may instantiate type arguments locally:

```
id(1)
Box::{ value: 1 }
Option::some(1)
```

1.17 Conditional Expressions

`if` is an expression:

```
if condition {
  then_value
} else {
  else_value
}
```

The `else` branch is mandatory.

Both branches must have the same type.

1.18 Match Expressions

`match` is an expression:

```
matchExpression:
    'match' expression '{' { matchArm } '}'

matchArm:
    pattern '=>' expression

pattern:
    '_'
  | valueName
  | literal
  | qualifiedVariantPattern
  | structPattern

qualifiedVariantPattern:
    typeName '::' variantName [ '(' [ pattern { ',' pattern } [ ',' ] ] ')' ]

structPattern:
    typeName '::' '{' structPatternField { ',' structPatternField } [ ',' ] '}'

structPatternField:
    fieldName
  | fieldName ':' pattern
```

```
match value {
  Option::none => fallback
  Option::some(x) => x
}
```

Match arms use `pattern => expression`.

Every match must be exhaustive.

V1 patterns:

- wildcard pattern: `_`,
- variable binding pattern: `name`,
- literal pattern,
- qualified enum variant pattern,
- qualified struct pattern.

Enum variants in patterns must be qualified:

```
Option::some(x)
Option::none
```

Bare identifiers in patterns are always variable bindings:

```
match value {
  x => x
}
```

Struct patterns use qualified syntax and must list all fields:

```
match p {
  Point::{ x, y } => x + y
}
```

Struct pattern punning is allowed. Explicit renaming is allowed:

```
match p {
  Point::{ x: a, y: b } => a + b
}
```

Rest patterns, spread patterns, guards, or-patterns, as-patterns, and `is` pattern expressions are not supported in v1.

1.19 Calls, Field Access, And Pipeline

Function call:

```
f(x, y)
```

Field access:

```
ops.add
```

Field access followed by call:

```
ops.add(x, y)
```

This is not method call syntax. Lane2 v1 has no method receiver lookup.

Pipeline syntax is supported:

```
value |> f(a, b)
```

It rewrites to:

```
f(value, a, b)
```

The right-hand side of `|>` must be a call or function literal.

The following are invalid:

```
value |> f
value |> f(_, y)
```

Placeholders are not supported.

1.20 Open Scope Extensions

`open value` opens a struct value.

The operand of `open` must be a visible value name with a struct type. It cannot be an arbitrary expression:

```
open ops
```

This is invalid:

```
open make_ops(10)
```

An open scope extension exposes the struct field values as unqualified names from the declaration point to the end of the current lexical scope.

At top level, `open` extends to the end of the file:

```
open int_add_ops

fn add_one(x : Int) -> Int {
  x + 1
}
```

Inside blocks, `open` is a local item and must appear before the final expression:

```
{
  open int_add_ops
  x + y
}
```

Local resolution uses the nearest preceding local binding or open scope extension, then falls back to preopen.

1.21 Preopen

The preopen namespace is a default-open namespace populated by anonymous top-level values.

```
let : Add[Int] = Add::{ add: int_add }
```

An anonymous top-level value must have a struct type. It exposes field values, not generated accessors.

Prelude-provided anonymous values are checked before user code, so their preopen entries are visible throughout user code.

User-defined anonymous top-level values extend preopen from their declaration point to the end of the top-level scope.

Preopen conflicts are errors.

Top-level `open` belongs to the global layer and conflicts with ordinary top-level names or preopened names. Local `open` may shadow preopen inside its lexical scope.

1.22 Struct Field Forwarding

A struct declaration may contain `open field` entries:

```
struct Compare[T] {
  equal_impl : Equal[T]
  open equal_impl
  less : (T, T) -> Bool
  less_eq : (T, T) -> Bool
  greater : (T, T) -> Bool
  greater_eq : (T, T) -> Bool
}
```

When a value of this struct type is opened, forwarded fields are exposed too.

Struct field forwarding affects only what is exposed by opening the containing struct value. It does not open the field inside ordinary function bodies.

1.23 Operators

Operators are aliases for recognized prelude operation fields.

There is no type-directed instance search. An operator is available only when the relevant operation field is available through preopen or an open scope extension.

Primitive operators are not special-cased. For example, `1 + 2` requires an available `Add::add` operation for `Int`.

Recognized operator mappings:

Operator	Operation
`+`	`Add::add`
`-`	`Sub::sub`

`*`	`Mul::mul`
`/`	`Div::div`
`%`	`Rem::rem`
unary `-`	`Neg::neg`
`==`	`Equal::equal`
`!=`	`Equal::not_equal`
`<`	`Compare::less`
`<=`	`Compare::less_eq`
`>`	`Compare::greater`
`>=`	`Compare::greater_eq`
`&&`	`And::and`
`||`	`Or::or`
`!`	`Not::not`

`&&` and `||` are strict operator aliases, not short-circuit control forms.

Concrete expression precedence, associativity, and unary/binary disambiguation follow the MoonBit parser reference where Lane2 has not deliberately removed a feature.

1.24 Unsafe Builtin

`builtin("...")` is an unsafe expression.

The checker does not interpret intrinsic strings. Instead, `builtin` receives its type from direct expected context:

```
fn int_add(a : Int, b : Int) -> Int {
  builtin("%i64_add")
}
```

This is valid because the function return type provides the expected type `Int` for the body expression.

This is invalid:

```
let x = builtin("%anything")
```

There is no direct expected type.

Incorrect builtin use can produce undefined behavior. Lane2's type safety guarantee applies only to programs that do not misuse `builtin`.

1.25 Trailing Commas

Comma-separated syntax lists allow trailing commas:

```
add(
  x,
  y,
)

Point::{
  x: 1,
  y: 2,
}
```

1.26 Deliberately Omitted From V1

V1 omits:

- mutation and assignment,
- field update and copy update,
- modules and imports,
- local type definitions,

- labeled function parameters,
- labeled enum payloads,
- type aliases,
- traits, typeclasses, interfaces,
- method syntax,
- short-circuit boolean operators,
- tuple types,
- collections,
- pattern guards,
- or-patterns,
- as-patterns,
- `is` pattern expressions,
- placeholder pipeline.